

Vault Cluster Setup Automation

Author: Prasadu Gamini

Objective

Complete automation solution for deploying HashiCorp Vault clusters on OpenShift/Kubernetes with support for unsealer vault pattern and transit auto-unseal.

Overview

This project automates the deployment of HashiCorp Vault clusters with the following features:

- **Multiple vault cluster support** (unsealer-vault + data vaults)
- **Transit auto-unseal** pattern for enhanced security
- **Automated initialization and unsealing** with built-in functions
- **TLS certificate generation** for secure communication
- **User policy management** with admin policy assignment
- **OpenShift Route creation** via Helm chart integration

Architecture

Unsealer Vault Pattern

- **Unsealer Vault:** Provides transit encryption for auto-unsealing data vaults
- **Data Vaults:** Use transit auto-unseal for automated unsealing
- **Benefits:** No manual unseal needed, enhanced security, scalability

Supported Components

- Prerequisites setup (hashicorp-vault-helm-pre-requisite)
- Vault cluster deployment (fndsec-hashicorp-vault-helm-1.6.0)
- Post-installation configuration (hashicorp-vault-post-install-helm-1.1.7)

Prerequisites

- Bash shell environment
- SSH access to Jenkins server (jenkins@ilceatm203)
- Remote machine must have:
 - OpenShift CLI (oc) configured
 - Helm 3.x installed
 - Access to OpenShift/Kubernetes cluster
 - jq for JSON parsing

Project Structure

```

vault-cluster-setup/
├── config-unsealer-vault.json      # Unsealer vault configuration
├── config-vault-1.json           # Data vault configuration
├── config.example.json           # Configuration template
├── setup-vault-cluster.sh        # Main orchestration script
├── create-namespace-route.sh     # Namespace creation (routes via Helm)
├── generate-certificates.sh      # TLS certificate generation
├── deploy-prerequisites.sh       # Prerequisites deployment
├── deploy-vault-cluster.sh       # Vault cluster deployment + init functions
├── deploy-post-install.sh        # Post-installation + user policy management
├── modules/
│   ├── logger.sh                # Bash logging utilities
│   ├── Logger.psm1              # PowerShell logging (legacy)
│   └── SSHConnection.psm1       # SSH connection (legacy)
└── logs/                        # Deployment logs (auto-created)

```

Configuration

Configuration Files

The project uses JSON configuration files for each vault cluster:

- **config-unsealer-vault.json**: Configuration for unsealer vault (transit encryption provider)
- **config-vault-1.json**: Configuration for data vault cluster
- **config.example.json**: Template for creating new configurations

Configuration Structure

```

{
  "directories": {
    "basePath": "/jenkins_home/vault-cluster-setup",
    "prerequisite": "hashicorp-vault-helm-pre-requisite",
    "cluster": "fndsec-hashicorp-vault-helm-1.6.0/unsealer-hashicorp-vault",
    "postInstall": "hashicorp-vault-post-install-helm-1.1.7"
  },
  "deployment": {
    "namespace": "unsealer-vault",
    "releaseName": "unsealer-hashicorp-vault",
    "routeName": "vault",
    "routeUrl": "vault-unsealer-
vault.apps.indelocpbmatm1059.ocpd.corp.amdocs.com",
    "helmTimeout": "10m"
  },
  "logging": {
    "logDir": "logs",
    "logLevel": "INFO"
  }
}

```

Key Configuration Options

- **basePath**: Base directory on remote server
- **namespace**: OpenShift/Kubernetes namespace for vault
- **releaseName**: Helm release name
- **routeUrl**: Custom OpenShift route hostname (optional)
- **helmTimeout**: Helm deployment timeout

Usage

Orchestration Script (setup-vault-cluster.sh)

The main orchestration script automates the entire Vault cluster setup process. It accepts multiple configuration files and processes each vault deployment sequentially.

Syntax:

```
./setup-vault-cluster.sh [OPTIONS] <CONFIG_FILES...>
```

Arguments:

- **CONFIG_FILES** - One or more configuration files (required)

Options:

- **-n, --skip-namespace** - Skip namespace and route creation
- **-s, --skip-certs** - Skip TLS certificate generation
- **-p, --skip-prereq** - Skip prerequisites installation
- **-d, --skip-deploy** - Skip Vault cluster deployment
- **-i, --skip-postinstall** - Skip post-installation tasks
- **-t, --test-connection** - Only test connection (oc cluster-info)
- **-h, --help** - Display help message

Examples:

```
# Deploy both unsealer vault and data vault
./setup-vault-cluster.sh config-unsealer-vault.json config-vault-1.json

# Deploy unsealer vault only
./setup-vault-cluster.sh config-unsealer-vault.json

# Deploy data vault only
./setup-vault-cluster.sh config-vault-1.json

# Test cluster connectivity
./setup-vault-cluster.sh -t config-unsealer-vault.json

# Skip certificate generation (if certificates already exist)
./setup-vault-cluster.sh -s config-unsealer-vault.json config-vault-1.json
```

```
# Skip prerequisites (if already deployed cluster-wide)
./setup-vault-cluster.sh -p config-vault-1.json

# Skip post-installation tasks
./setup-vault-cluster.sh -i config-unsealer-vault.json config-vault-1.json
```

Processing Order:

1. Validates all config files
2. Creates namespaces/routes for all vaults (once)
3. Generates TLS certificates for each vault
4. Deploys prerequisites (once, cluster-wide)
5. Deploys each vault cluster sequentially
6. Runs post-installation for each vault
7. Displays status for all deployed vaults

Individual Deployment Steps

1. Create Namespaces

```
# Creates namespaces (routes now created by Helm chart)
./create-namespace-route.sh config-unsealer-vault.json config-vault-1.json
```

2. Generate TLS Certificates

```
# Generate certificates for each vault cluster
./generate-certificates.sh config-unsealer-vault.json
./generate-certificates.sh config-vault-1.json
```

3. Deploy Prerequisites

```
# Prerequisites are cluster-wide and only need to be deployed once
# Uses config-bash.json by default (no parameters needed)
./deploy-prerequisites.sh
```

4. Deploy Vault Cluster

```
# Deploys vault cluster and creates OpenShift route via Helm
./deploy-vault-cluster.sh config-unsealer-vault.json config-vault-1.json
```

5. Initialize and Unseal Vaults

Use the built-in functions in `deploy-vault-cluster.sh`:

```
# Example script to initialize unsealer vault
source modules/logger.sh
initialize_logger "logs" "INFO"

# Initialize unsealer vault
initialize_unsealer_vault "unsealer-vault" "$LOG_FILE" "/tmp/vault-init-keys.json"

# Enable basic features
enable_vault_features "unsealer-vault" "$LOG_FILE" "/tmp/vault-init-keys.json"

# Setup transit auto-unseal for vault-1
setup_transit_autounseal "unsealer-vault" "vault-1" "$LOG_FILE" "/tmp/vault-init-keys.json" "autounseal_1"

# Initialize data vault with transit auto-unseal
initialize_data_vault "vault-1" "$LOG_FILE" "/tmp/vault1-init-keys.json"

# Enable features on data vault
enable_vault_features "vault-1" "$LOG_FILE" "/tmp/vault1-init-keys.json"
```

6. Deploy Post-Installation

```
./deploy-post-install.sh config-unsealer-vault.json
./deploy-post-install.sh config-vault-1.json
```

7. Update User Policies (Optional)

Use the built-in function to grant admin access:

```
# Example: Update user policies after post-install
source modules/logger.sh
initialize_logger "logs" "INFO"

ROOT_TOKEN="<your-root-token>"
update_user_admin_policies "vault-1" "$ROOT_TOKEN" "$LOG_FILE"
```

Deployment Steps

0. Pre-Deployment Requirements

Required Files to be Staged on Remote Server:

Before running the deployment scripts, ensure all required Helm charts and configuration files are present on the remote server:

```
/jenkins_home/vault-cluster-setup/
├─ hashicorp-vault-helm-pre-requisite/
│   └─ fndsec-hashicorp-vault-helm-pre-requisite-1.6.0.tgz  # Pre-requisite Helm
chart
│   └─ custom-values.yaml                                # Custom values for
pre-req
│
├─ fndsec-hashicorp-vault-helm-1.6.0/
│   └─ unsealer-hashicorp-vault/
│       └─ fndsec-hashicorp-vault-helm-1.6.0.tgz          # Unsealer vault Helm
chart
│   └─ custom-values.yaml                                # Custom values for
unsealer
│   └─ values.openshift.yaml                            # OpenShift-specific
values
│       └─ hashicorp-vault/
│           └─ fndsec-hashicorp-vault-helm-1.6.0.tgz      # Data vault Helm
chart
│       └─ custom-values.yaml                            # Custom values for
data vault
│       └─ values.openshift.yaml                        # OpenShift-specific
values
└─ hashicorp-vault-post-install-helm-1.1.7/
    └─ fndsec-hashicorp-vault-post-install-helm-1.1.7.tgz # Post-install Helm
chart
```

Verification:

```
# Check all required files are present
find /jenkins_home/vault-cluster-setup -type f \( -name 'custom-values.yaml' -o -
name 'values.openshift.yaml' -o -name '*.tgz' \)
```

Key Files:

- **.tgz files:** Packaged Helm charts for deployment
- **custom-values.yaml:** Custom configuration values (specific to your environment)
- **values.openshift.yaml:** OpenShift-specific configurations (routes, security contexts)

1. Namespace Creation

- Creates OpenShift/Kubernetes namespaces
- **Route creation disabled** - routes are now created by Helm chart during cluster deployment
- Validates namespace creation

2. TLS Certificate Generation

- Generates CA certificate and private key (4096-bit RSA)
- Creates server certificates with proper SANs for Kubernetes
- Includes all Vault service DNS names
- Creates Kubernetes TLS secrets automatically
- Copies certificates to cluster directory
- Validity: 10 years

Certificate SANs include:

- `vault, vault.<namespace>, vault.<namespace>.svc.cluster.local`
- `vault-0.vault-internal.<namespace>.svc.cluster.local`
- `vault-1.vault-internal.<namespace>.svc.cluster.local`
- `vault-2.vault-internal.<namespace>.svc.cluster.local`
- `vault-active.<namespace>.svc.cluster.local`
- `localhost, 127.0.0.1`

3. Prerequisites Deployment

- Installs required Kubernetes resources
- Sets up secrets and ConfigMaps
- Configures storage classes
- Deploys supporting services

4. Vault Cluster Deployment

- Deploys Vault StatefulSet (3 replicas with Raft storage)
- Configures TLS certificates
- Sets up Vault services
- **Creates OpenShift Route** via Helm chart (passthrough TLS)
- Configures high availability with Raft consensus
- **Validates required secrets** before deployment

Secret validation:

- For unsealer-vault: Only checks for `vault-server-tls` secret
- For data vaults: Checks for both `vault-server-tls` AND `vault-transit-token-secret`
- Deployment fails if required secrets are missing
- Provides clear error messages with remediation steps

Route creation:

- Routes are automatically created by Helm using `--set server.route.enabled=true`
- Custom hostname via `--set server.route.host=<routeUrl>`
- Passthrough TLS termination
- Targets `vault-active` service

5. Vault Initialization (NEW Functions!)

Built-in automation functions in deploy-vault-cluster.sh:**initialize_unsealer_vault(namespace, log_file, init_file)**

- Initializes vault with 5 key shares, 3 threshold
- Unseals all 3 vault pods automatically
- Logs in with root token
- Verifies Raft cluster peers
- Saves keys to JSON file

setup_transit_auto_unseal(unsealer_ns, data_ns, log_file, init_file, transit_key)

- Enables transit secrets engine on unsealer vault
- Creates transit encryption key
- Generates auto-unseal policy and token
- Creates **vault-transit-token-secret** in data vault namespace
- Configures transit auto-unseal for data vaults

initialize_data_vault(namespace, log_file, init_file)

- Initializes data vault with recovery keys (transit auto-unseal)
- 5 recovery key shares, 3 threshold
- Auto-unsealed via transit engine
- Saves recovery keys and root token

enable_vault_features(namespace, log_file, init_file)

- Enables KV v2 secrets engine at path **/secret**
- Extensible for additional features

6. Post-Installation Configuration**Dynamic configuration based on namespace:**

The deploy-post-install.sh script automatically detects the vault type (unsealer vs data vault) and configures the post-installation accordingly:

For unsealer-vault:

- Sets **deploymentMode: unsealerVaultClusterSetup**
- Sets **vaultClusterNamespaces: vault-1** (managed data vaults)
- Skips user policy updates (not applicable)

For data vaults (vault-1, vault-2, etc.):

- Sets **deploymentMode: vaultClusterSetup**
- Sets **unsealerVaultUrl: https://vault-active.unsealer-vault.svc.cluster.local:8200**
- Updates user admin policies (if root token provided)

Common Helm chart approach:

- Both vault types use the same Helm chart package (.tgz)
- Values are dynamically overridden using --set flags
- No need for separate post-install directories

Configuration applied:

- Applies Helm post-install jobs
- Configures Kubernetes authentication
- Creates admin policies and roles
- Sets up userpass authentication
- Creates vault-secrets-migration-user and vault-secrets-management-user
- Enables audit logging

7. User Policy Management (NEW Feature!)

Automated user policy updates for data vaults:

The deploy-post-install.sh script can automatically update user policies when provided with a root token.

Usage:

```
# Method 1: Pass root token as argument
./deploy-post-install.sh config-vault-1.json hvs.xxxxxxxxxxxxxx

# Method 2: Use environment variable
export VAULT_ROOT_TOKEN=hvs.xxxxxxxxxxxxxx
./deploy-post-install.sh config-vault-1.json
```

What it does:

- ☒ Only runs for data vaults (skips unsealer-vault)
- ☒ Checks if Vault is initialized
- ☒ Verifies vault-admin-policy exists
- ☒ Checks if userpass auth is enabled
- ☒ Updates both users from sm-secret-policy to vault-admin-policy:
 - vault-secrets-migration-user
 - vault-secrets-management-user
- ☒ Grants full admin access: create, read, update, delete, list, sudo on all paths
- ☒ Enables full Vault UI access
- ☒ Verifies policy updates

Manual policy update (if needed):

```
ROOT_TOKEN="hvs.xxxxxxxxxxxxxx"

oc exec vault-0 -n vault-1 -- sh -c "export VAULT_TOKEN=$ROOT_TOKEN ; \
  vault write auth/userpass/users/vault-secrets-migration-user policies='vault-
admin-policy' ; \
```

```

vault write auth/userpass/users/vault-secrets-management-user policies='vault-admin-policy'

```

Remote Directories

The automation works with these directories on jenkins@ilceatm203:

```

/jenkins_home/vault-cluster-setup/
├─ hashicorp-vault-helm-pre-requisite/
├─ fndsec-hashicorp-vault-helm-1.6.0/
│   └─ unsealer-hashicorp-vault/
│       └─ hashicorp-vault/
├─ hashicorp-vault-post-install-helm-1.1.7/
│   └─ fndsec-hashicorp-vault-post-install-helm/ # Common Helm chart (used by
all vaults)
│       └─ fndsec-hashicorp-vault-post-install-helm-1.1.7.tgz # Packaged chart
├─ unsealer-vault-certs/
└─ vault-1-certs/

```

Note: Both unsealer-vault and data vaults use the same post-install Helm chart. The deploy-post-install.sh script dynamically configures values based on the namespace.

Certificate Management

Automatic TLS Certificate Generation

The script automatically generates TLS certificates for each vault cluster:

- ☒ **Generates CA certificate** (4096-bit RSA)
- ☒ **Creates server certificates** with proper SANs for Kubernetes
- ☒ **Includes all service DNS names** (vault, vault-0, vault-1, vault-2, vault-active, vault-internal)
- ☒ **Creates Kubernetes TLS secret** (`vault-server-tls`) in vault namespace
- ☒ **Copies to cluster directory** automatically
- ☒ **10-year validity** period

Certificate SANs include:

- `vault, vault.<namespace>, vault.<namespace>.svc.cluster.local`
- `vault-0.vault-internal.<namespace>.svc.cluster.local`
- `vault-1.vault-internal.<namespace>.svc.cluster.local`
- `vault-2.vault-internal.<namespace>.svc.cluster.local`
- `vault-active.<namespace>.svc.cluster.local`
- `vault-internal.<namespace>.svc.cluster.local`
- `localhost, 127.0.0.1`

Generated files per vault cluster:

- `/jenkins_home/vault-cluster-setup/<namespace>-certs/ca.crt` - CA certificate
- `/jenkins_home/vault-cluster-setup/<namespace>-certs/vault.crt` - Server certificate

- `/jenkins_home/vault-cluster-setup/<namespace>-certs/vault.key` - Private key

Kubernetes Secret:

```
# View certificate
oc get secret vault-server-tls -n <namespace> -o jsonpath='{.data.tls\.crt}' |
base64 -d | openssl x509 -text
```

Required Secrets for Deployment

The `deploy-vault-cluster.sh` script validates that required secrets exist before deployment:

For Unsealer Vault

Required:

- `vault-server-tls` - TLS certificates (created by `generate-certificates.sh`)

For Data Vaults (`vault-1`, `vault-2`, etc.)

Required:

- `vault-server-tls` - TLS certificates (created by `generate-certificates.sh`)
- `vault-transit-token-secret` - Transit token for auto-unseal (created by `setup_transit_autounseal` function)

How secrets are created:

1. **`vault-server-tls`** - Created automatically by `generate-certificates.sh`:

```
./generate-certificates.sh config-unsealer-vault.json
./generate-certificates.sh config-vault-1.json
```

2. **`vault-transit-token-secret`** - Created by `setup_transit_autounseal()` function:

```
# After unsealer vault is initialized and unsealed
setup_transit_autounseal "unsealer-vault" "vault-1" "$LOG_FILE" \
    "/tmp/vault-init-keys.json" "autounseal_1"
```

Or manually:

```
# Create transit token on unsealer vault first, then:
oc create secret generic vault-transit-token-secret \
    --from-literal=token="<transit-token>" \
    -n vault-1
```

Validation behavior:

- Script checks for secrets before Helm deployment
- For unsealer-vault: Skips vault-transit-token-secret check
- For data vaults: Requires both secrets
- Deployment fails with clear error if secrets are missing
- Error message includes commands to create missing secrets

OpenShift Route Configuration

Automatic OpenShift Route Creation via Helm

Routes are now created automatically by the Helm chart during vault cluster deployment:

Key Features:

- ☒ **Created by Helm** during `deploy-vault-cluster.sh`
- ☒ **Passthrough TLS** termination (Vault handles TLS)
- ☒ **Custom hostname** support via `routeUrl` in config
- ☒ **Service Target:** `vault-active` (active Vault node)
- ☒ **Port:** 8200 (HTTPS)

Deployment: Routes are created via Helm flags:

```
helm upgrade --install <release> <chart> \  
  --set server.route.enabled=true \  
  --set server.route.host=<routeUrl>
```

Route Specifications:

- **Service Target:** `vault-active` (active Vault node)
- **Port:** 8200 (HTTPS)
- **TLS Mode:** Passthrough
- **Insecure Traffic:** Disabled

Accessing Vault via Route:

```
# Get the route URL  
oc get route vault -n <namespace> -o jsonpath='{.spec.host}'  
  
# Or from config  
# unsealer-vault: vault-unsealer-vault.apps.indelocpbmatm1059.ocpd.corp.amdocs.com  
# vault-1: vault-vault-1.apps.indelocpbmatm1059.ocpd.corp.amdocs.com  
  
# Access Vault UI/API  
curl -k https://<route-hostname>/v1/sys/health  
  
# Vault UI  
https://<route-hostname>/ui/
```

Manual Route Creation (if needed):

```
# Create route directly
oc create route passthrough vault \
  --service=vault-active \
  --port=8200 \
  --hostname=<custom-hostname> \
  --namespace=<namespace>
```

Logging

All deployment activities are logged to:

- Console output (color-coded)
- Log files in `logs/` directory
- Format: `vault_setup_YYYYMMDD_HHmss.log`

Log levels: DEBUG, INFO, WARN, ERROR

Post-Deployment

After successful deployment, the vaults are ready for initialization and configuration.

Unsealer Vault Setup

1. Initialize Unsealer Vault (automated via functions)

```
# Using built-in function
initialize_unsealer_vault "unsealer-vault" "$LOG_FILE" "/tmp/vault-init-
keys.json"
```

Or manually:

```
oc exec vault-0 -n unsealer-vault -- vault operator init \
  -key-shares=5 -key-threshold=3 -format=json | tee /tmp/vault-init-
keys.json
```

Save the unseal keys and root token securely!

2. Unseal Unsealer Vault (automated via function or manual)

```
# Extract keys
KEY1=$(jq -r '.unseal_keys_b64[0]' /tmp/vault-init-keys.json)
KEY2=$(jq -r '.unseal_keys_b64[1]' /tmp/vault-init-keys.json)
```

```
KEY3=$(jq -r '.unseal_keys_b64[2]' /tmp/vault-init-keys.json)

# Unseal all pods
for pod in vault-0 vault-1 vault-2; do
  oc exec $pod -n unsealer-vault -- vault operator unseal $KEY1
  oc exec $pod -n unsealer-vault -- vault operator unseal $KEY2
  oc exec $pod -n unsealer-vault -- vault operator unseal $KEY3
done
```

3. Enable Transit Auto-Unseal (automated via function)

```
setup_transit_automseal "unsealer-vault" "vault-1" "$LOG_FILE" \
  "/tmp/vault-init-keys.json" "automseal_1"
```

This automatically:

- Enables transit secrets engine
- Creates transit encryption key
- Creates automseal policy and token
- Creates vault-transit-token-secret in vault-1 namespace

Data Vault Setup (vault-1)

1. Initialize Data Vault with Transit Auto-Unseal (automated)

```
initialize_data_vault "vault-1" "$LOG_FILE" "/tmp/vault1-init-keys.json"
```

Or manually:

```
oc exec vault-0 -n vault-1 -- vault operator init \
  -recovery-shares=5 -recovery-threshold=3 -format=json | tee /tmp/vault1-
init-keys.json
```

Note: With transit auto-unseal, vault automatically unseals. No manual unseal needed!

2. Enable Basic Features (automated)

```
enable_vault_features "vault-1" "$LOG_FILE" "/tmp/vault1-init-keys.json"
```

3. Run Post-Installation

```
./deploy-post-install.sh config-vault-1.json
```

4. Update User Policies (automated)

```
ROOT_TOKEN=$(jq -r '.root_token' /tmp/vault1-init-keys.json)
update_user_admin_policies "vault-1" "$ROOT_TOKEN" "$LOG_FILE"
```

This grants both users full admin access:

- vault-secrets-migration-user
- vault-secrets-management-user

Check Status

```
# Check unsealer vault
oc exec vault-0 -n unsealer-vault -- vault status

# Check data vault (auto-unsealed)
oc exec vault-0 -n vault-1 -- vault status

# Check Raft peers
ROOT_TOKEN=$(jq -r '.root_token' /tmp/vault-init-keys.json)
oc exec vault-0 -n unsealer-vault -- env VAULT_TOKEN=$ROOT_TOKEN vault operator
raft list-peers
```

Access Vault UI

```
# Get route URLs
oc get route vault -n unsealer-vault -o jsonpath='{.spec.host}'
oc get route vault -n vault-1 -o jsonpath='{.spec.host}'

# Access Vault UI
# Unsealer: https://vault-unsealer-
vault.apps.indelocpbmatm1059.ocpd.corp.amdocs.com/ui/
# Vault-1: https://vault-vault-1.apps.indelocpbmatm1059.ocpd.corp.amdocs.com/ui/
```

Login credentials:

- **Root token:** From init files
- **vault-secrets-management-user:** Check secret `vault-secrets-management-user-secret`
- **vault-secrets-migration-user:** Check secret `vault-secrets-migration-user-secret`

```
# Get user passwords
oc get secret vault-secrets-management-user-secret -n vault-1 -o
jsonpath='{.data.password}' | base64 -d
```

```
oc get secret vault-secrets-migration-user-secret -n vault-1 -o
jsonpath='{.data.password}' | base64 -d
```

Troubleshooting

Pod Status Issues

```
# Check pod status
oc get pods -n unsealer-vault
oc get pods -n vault-1

# Get pod logs
oc logs -n unsealer-vault vault-0
oc logs -n vault-1 vault-0

# Describe pod for events
oc describe pod -n unsealer-vault vault-0
```

Vault Sealed Issues

```
# Check vault status
oc exec vault-0 -n unsealer-vault -- vault status

# For unsealer vault: manually unseal
oc exec vault-0 -n unsealer-vault -- vault operator unseal <key>

# For data vault: check transit token secret
oc get secret vault-transit-token-secret -n vault-1
```

Helm Deployment Failures

```
# Check Helm releases
helm list -n unsealer-vault
helm list -n vault-1

# Get release details
helm status vault -n unsealer-vault
helm get values vault -n unsealer-vault
```

Certificate Issues

```
# Verify TLS secrets
oc get secrets vault-server-tls -n unsealer-vault
oc get secrets vault-server-tls -n vault-1
```



```
# Check certificate validity
oc get secret vault-server-tls -n unsealer-vault -o jsonpath='{.data.tls\.crt}' | \
  base64 -d | openssl x509 -text -noout

# Verify certificate SANs
oc get secret vault-server-tls -n unsealer-vault -o jsonpath='{.data.tls\.crt}' | \
  base64 -d | openssl x509 -text -noout | grep -A1 "Subject Alternative Name"
```

Route Issues

```
# Check route status
oc get route vault -n unsealer-vault
oc describe route vault -n unsealer-vault

# Test route connectivity
curl -k https://$(oc get route vault -n unsealer-vault -o \
  jsonpath='{.spec.host}')/v1/sys/health
```

Post-Install Job Issues

```
# Check post-install job status
oc get jobs -n vault-1 | grep post-install
oc get pods -n vault-1 | grep post-install

# Get job logs
oc logs -n vault-1 <post-install-job-pod>

# Delete and retry post-install
helm uninstall vault-1-post-install -n vault-1
oc delete job vault-1-post-install-job -n vault-1
./deploy-post-install.sh config-vault-1.json
```

Transit Auto-Unseal Issues

```
# Check unsealer vault transit engine
ROOT_TOKEN=$(jq -r '.root_token' /tmp/vault-init-keys.json)
oc exec vault-0 -n unsealer-vault -- env VAULT_TOKEN=$ROOT_TOKEN vault secrets \
  list

# Verify transit key exists
oc exec vault-0 -n unsealer-vault -- env VAULT_TOKEN=$ROOT_TOKEN vault list \
  transit/keys
```

```
# Check transit token secret in data vault
oc get secret vault-transit-token-secret -n vault-1 -o yaml
```

User Policy Issues

```
# Check if users exist
oc exec vault-0 -n vault-1 -- env VAULT_TOKEN=$ROOT_TOKEN vault list
auth/userpass/users

# Check user policies
oc exec vault-0 -n vault-1 -- env VAULT_TOKEN=$ROOT_TOKEN \
  vault read auth/userpass/users/vault-secrets-management-user

# Check available policies
oc exec vault-0 -n vault-1 -- env VAULT_TOKEN=$ROOT_TOKEN vault policy list

# Read policy content
oc exec vault-0 -n vault-1 -- env VAULT_TOKEN=$ROOT_TOKEN \
  vault policy read vault-admin-policy
```

Logging

All deployment activities are logged to:

- Console output with INFO/WARN/ERROR levels
- Log files in `logs/` directory
- Format: `vault_<operation>_<namespace>_YYYYMMDD_HH:mm:ss.log`

Log levels: DEBUG, INFO, WARN, ERROR

View logs:

```
# List recent logs
ls -lt logs/ | head -10

# Follow a specific log
tail -f logs/vault_cluster_deployment_unsealer-vault_*.log

# Search for errors
grep -i error logs/*.log
grep -i warn logs/*.log
```

Security Notes

- **Store init files securely** (contain unseal keys and root tokens)
 - `/tmp/vault-init-keys.json` (unsealer vault)
 - `/tmp/vault1-init-keys.json` (data vault)

- Use SSH key authentication instead of passwords when possible
- Rotate passwords regularly
- **Never commit credentials to version control**
- Store Vault unseal keys in secure location (KMS, HSM, or secure vault)
- Use transit auto-unseal for production (eliminates manual unsealing)
- Regularly backup Vault data
- Monitor audit logs
- Implement proper RBAC policies

Best Practices

High Availability

- Deploy 3+ vault pods for HA
- Use Raft storage backend for consensus
- Monitor Raft cluster health
- Test failover scenarios

Security

- Enable audit logging on all vaults
- Use transit auto-unseal for data vaults
- Implement least privilege policies
- Regularly rotate tokens and credentials
- Enable MFA for sensitive operations

Monitoring

- Monitor vault seal status
- Track Raft cluster health
- Monitor certificate expiration
- Set up alerts for critical events

Backup & Recovery

- Backup Raft snapshots regularly
- Test restore procedures
- Document recovery procedures
- Store backups securely

Support

For issues or questions:

1. Check log files in `logs/` directory
2. Review deployment output
3. Verify OpenShift cluster health
4. Check Helm releases and pod status
5. Review troubleshooting section

License

Internal use only - HashiCorp Vault Cluster Setup